

名为 Chapter6.h 的头文件中。显然与其他所有用到 fact 函数的文件一样, fact.cc 应该包含 Chapter6.h 头文件。另外,我们在名为 factMain.cc 的文件中创建 main 函数,main 函数将调用 fact 函数。要生成可执行文件 (executable file), 必须告诉编译器我们用到的代码在哪里。对于上述几个文件来说,编译的过程如下所示:

```
$ CC factMain.cc fact.cc # generates factMain.exe or a.out
$ CC factMain.cc fact.cc -o main # generates main or main.exe
```

其中,CC 是编译器的名字,\$是系统提示符,#后面是命令行下的注释语句。接下来运行可执行文件,就会执行我们定义的 main 函数。

如果我们修改了其中一个源文件,那么只需重新编译那个改动了的文件。大多数编译器提供了分离式编译每个文件的机制,这一过程通常会产生一个后缀名是 .obj (Windows) 或 .o (UNIX) 的文件,后缀名的含义是该文件包含对象代码 (object code)。

接下来编译器负责把对象文件链接在一起形成可执行文件。在我们的系统中,编译的过程如下所示:

```
$ CC -c factMain.cc # generates factMain.o
$ CC -c fact.cc # generates fact.o
$ CC factMain.o fact.o # generates factMain.exe or a.out
$ CC factMain.o fact.o -o main # generates main or main.exe
```

你可以仔细阅读编译器的用户手册,弄清楚由多个文件组成的程序是如何编译并执行的。

6.1.3 节练习

练习 6.9: 编写你自己的 fact.cc 和 factMain.cc,这两个文件都应该包含上一小节的练习中编写的 Chapter6.h 头文件。通过这些文件,理解你的编译器是如何支持分离式编译的。

6.2 参数传递



如前所述,每次调用函数时都会重新创建它的形参,并用传入的实参对形参进行初始化。



形参初始化的机理与变量初始化一样。

和其他变量一样,形参的类型决定了形参和实参交互的方式。如果形参是引用类型(参见 2.3.1 节,第 45 页),它将绑定到对应的实参上;否则,将实参的值拷贝后赋给形参。

当形参是引用类型时,我们说它对应的实参被**引用传递** (passed by reference) 或者函数被**传引用调用** (called by reference)。和其他引用一样,引用形参也是它绑定的对象的别名;也就是说,引用形参是它对应的实参的别名。

当实参的值被拷贝给形参时,形参和实参是两个相互独立的对象。我们说这样的实参被**值传递** (passed by value) 或者函数被**传值调用** (called by value)。

6.2.1 传值参数



当初始化一个非引用类型的变量时,初始值被拷贝给变量。此时,对变量的改动不会

影响初始值:

```
int n = 0;           // int 类型的初始变量
int i = n;           // i 是 n 的值的副本
i = 42;              // i 的值改变; n 的值不变
```

传值参数的机理完全一样, 函数对形参做的所有操作都不会影响实参。例如, 在 fact 函数 (参见 6.1 节, 第 182 页) 内对变量 val 执行递减操作:

```
ret *= val--;        // 将 val 的值减 1
```

尽管 fact 函数改变了 val 的值, 但是这个改动不会影响传入 fact 的实参。调用 fact(i) 不会改变 i 的值。

指针形参

指针的行为和其他非引用类型一样。当执行指针拷贝操作时, 拷贝的是指针的值。拷贝之后, 两个指针是不同的指针。因为指针使我们可以间接地访问它所指的对象, 所以通过指针可以修改它所指对象的值:

```
int n = 0, i = 42;
int *p = &n, *q = &i;    // p 指向 n; q 指向 i
*p = 42;                  // n 的值改变; p 不变
p = q;                    // p 现在指向了 i; 但是 i 和 n 的值都不变
```

指针形参的行为与之类似:

```
// 该函数接受一个指针, 然后将指针所指的值为 0
void reset(int *ip)
{
    *ip = 0;    // 改变指针 ip 所指对象的值
    ip = 0;     // 只改变了 ip 的局部拷贝, 实参未被改变
}
```

调用 reset 函数之后, 实参所指的对象被置为 0, 但是实参本身并没有改变:

```
int i = 42;
reset(&i);    // 改变 i 的值而非 i 的地址
cout << "i = " << i << endl;    // 输出 i = 0
```

210

Best Practices

熟悉 C 的程序员常常使用指针类型的形参访问函数外部的对象。在 C++ 语言中, 建议使用引用类型的形参替代指针。

6.2.1 节练习

练习 6.10: 编写一个函数, 使用指针形参交换两个整数的值。在代码中调用该函数并输出交换后的结果, 以此验证函数的正确性。



6.2.2 传引用参数

回忆过去所学的知识, 我们知道对于引用的操作实际上是作用在引用所引的对象上 (参见 2.3.1 节, 第 45 页):

```
int n = 0, i = 42;
int &r = n;    // r 绑定了 n (即 r 是 n 的另一个名字)
```

```
r = 42;           // 现在 n 的值是 42
r = i;           // 现在 n 的值和 i 相同
i = r;           // i 的值和 n 相同
```

引用形参的行为与之类似。通过使用引用形参，允许函数改变一个或多个实参的值。

举个例子，我们可以改写上一小节的 `reset` 程序，使其接受的参数是引用类型而非指针：

```
//该函数接受一个 int 对象的引用，然后将对象的值置为 0
void reset(int &i)    // i 是传给 reset 函数的对象的另一个名字
{
    i = 0;           // 改变了 i 所引对象的值
}
```

和其他引用一样，引用形参绑定初始化它的对象。当调用这一版本的 `reset` 函数时，`i` 绑定我们传给函数的 `int` 对象，此时改变 `i` 也就是改变 `i` 所引对象的值。此例中，被改变的对象是传入 `reset` 的实参。

调用这一版本的 `reset` 函数时，我们直接传入对象而无须传递对象的地址：

```
int j = 42;
reset(j);           // j 采用传引用方式，它的值被改变
cout << "j = " << j << endl;    // 输出 j = 0
```

在上述调用过程中，形参 `i` 仅仅是 `j` 的又一个名字。在 `reset` 内部对 `i` 的使用即是对 `j` 的使用。

使用引用避免拷贝

◀ 211

拷贝大的类类型对象或者容器对象比较低效，甚至有的类类型（包括 `IO` 类型在内）根本就不支持拷贝操作。当某种类型不支持拷贝操作时，函数只能通过引用形参访问该类型的对象。

举个例子，我们准备编写一个函数比较两个 `string` 对象的长度。因为 `string` 对象可能会非常长，所以应该尽量避免直接拷贝它们，这时使用引用形参是比较明智的选择。又因为比较长度无须改变 `string` 对象的内容，所以把形参定义成对常量的引用（参见 2.4.1 节，第 54 页）：

```
// 比较两个 string 对象的长度
bool isShorter(const string &s1, const string &s2)
{
    return s1.size() < s2.size();
}
```

如 6.2.3 节（第 191 页）将要介绍的，当函数无须修改引用形参的值时最好使用常量引用。

Best
Practices

如果函数无须改变引用形参的值，最好将其声明为常量引用。

使用引用形参返回额外信息

一个函数只能返回一个值，然而有时函数需要同时返回多个值，引用形参为我们一次返回多个结果提供了有效的途径。举个例子，我们定义一个名为 `find_char` 的函数，它返回在 `string` 对象中某个指定字符第一次出现的位置。同时，我们也希望函数能返回该

字符出现的总次数。

该如何定义函数使得它能够既返回位置也返回出现次数呢？一种方法是定义一个新的数据类型，让它包含位置和数量两个成员。还有另一种更简单的方法，我们可以给函数传入一个额外的引用实参，令其保存字符出现的次数：

```
// 返回 s 中 c 第一次出现的位置索引
// 引用形参 occurs 负责统计 c 出现的总次数
string::size_type find_char(const string &s, char c,
                           string::size_type &occurs)
{
    auto ret = s.size();           // 第一次出现的位置（如果有的话）
    occurs = 0;                   // 设置表示出现次数的形参的值
    for (decltype(ret) i = 0; i != s.size(); ++i) {
        if (s[i] == c) {
            if (ret == s.size())
                ret = i;           // 记录 c 第一次出现的位置
            ++occurs;              // 将出现的次数加 1
        }
    }
    return ret;                   // 出现次数通过 occurs 隐式地返回
}
```

212 当我们调用 `find_char` 函数时，必须传入三个实参：作为查找范围的一个 `string` 对象、要找的字符以及一个用于保存字符出现次数的 `size_type`（参见 3.2.2 节，第 79 页）对象。假设 `s` 是一个 `string` 对象，`ctr` 是一个 `size_type` 对象，则我们通过如下形式调用 `find_char` 函数：

```
auto index = find_char(s, 'o', ctr);
```

调用完成后，如果 `string` 对象中确实存在 `o`，那么 `ctr` 的值就是 `o` 出现的次数，`index` 指向 `o` 第一次出现的位置；否则如果 `string` 对象中没有 `o`，`index` 等于 `s.size()` 而 `ctr` 等于 0。

6.2.2 节练习

练习 6.11：编写并验证你自己的 `reset` 函数，使其作用于引用类型的参数。

练习 6.12：改写 6.2.1 节中练习 6.10（第 188 页）的程序，使用引用而非指针交换两个整数的值。你觉得哪种方法更易于使用呢？为什么？

练习 6.13：假设 `T` 是某种类型的名字，说明以下两个函数声明的区别：一个是 `void f(T)`，另一个是 `void f(&T)`。

练习 6.14：举一个形参应该是引用类型的例子，再举一个形参不能是引用类型的例子。

练习 6.15：说明 `find_char` 函数中的三个形参为什么是现在的类型，特别说明为什么 `s` 是常量引用而 `occurs` 是普通引用？为什么 `s` 和 `occurs` 是引用类型而 `c` 不是？如果令 `s` 是普通引用会发生什么情况？如果令 `occurs` 是常量引用会发生什么情况？



6.2.3 const 形参和实参

当形参是 `const` 时，必须要注意 2.4.3 节（第 57 页）关于顶层 `const` 的讨论。如前

所述，顶层 const 作用于对象本身：

```
const int ci = 42;    // 不能改变 ci, const 是顶层的
int i = ci;          // 正确：当拷贝 ci 时，忽略了它的顶层 const
int * const p = &i;  // const 是顶层的，不能给 p 赋值
*p = 0;              // 正确：通过 p 改变对象的内容是允许的，现在 i 变成了 0
```

和其他初始化过程一样，当用实参初始化形参时会忽略掉顶层 const。换句话说，形参的顶层 const 被忽略掉了。当形参有顶层 const 时，传给它常量对象或者非常量对象都是可以的：

```
void fcn(const int i) { /* fcn 能够读取 i，但是不能向 i 写值 */ }
```

调用 fcn 函数时，既可以传入 const int 也可以传入 int。忽略掉形参的顶层 const 可能产生意想不到的结果：

```
void fcn(const int i) { /* fcn 能够读取 i，但是不能向 i 写值 */ }
void fcn(int i) { /* ... */ } // 错误：重复定义了 fcn(int)
```

213

在 C++ 语言中，允许我们定义若干具有相同名字的函数，不过前提是不同函数的形参列表应该有明显的区别。因为顶层 const 被忽略掉了，所以在上面的代码中传入两个 fcn 函数的参数可以完全一样。因此第二个 fcn 是错误的，尽管形式上有差异，但实际上它的形参和第一个 fcn 的形参没什么不同。

指针或引用形参与 const

形参的初始化方式和变量的初始化方式是一样的，所以回顾通用的初始化规则有助于理解本节知识。我们可以使用非常量初始化一个底层 const 对象，但是反过来不行；同时一个普通的引用必须用同类型的对象初始化。

```
int i = 42;
const int *cp = &i; // 正确：但是 cp 不能改变 i（参见 2.4.2 节，第 56 页）
const int &r = i;    // 正确：但是 r 不能改变 i（参见 2.4.1 节，第 55 页）
const int &r2 = 42;  // 正确：（参见 2.4.1 节，第 55 页）
int *p = cp;        // 错误：p 的类型和 cp 的类型不匹配（参见 2.4.2 节，第 56 页）
int &r3 = r;         // 错误：r3 的类型和 r 的类型不匹配（参见 2.4.1 节，第 55 页）
int &r4 = 42;        // 错误：不能用字面值初始化一个非常量引用（参见 2.3.1 节，第 45 页）
```

将同样的初始化规则应用到参数传递上可得如下形式：

```
int i = 0;
const int ci = i;
string::size_type ctr = 0;
reset(&i);          // 调用形参类型是 int* 的 reset 函数
reset(&ci);          // 错误：不能用指向 const int 对象的指针初始化 int*
reset(i);           // 调用形参类型是 int& 的 reset 函数
reset(ci);          // 错误：不能把普通引用绑定到 const 对象 ci 上
reset(42);          // 错误：不能把普通应用绑定到字面值上
reset(ctr);         // 错误：类型不匹配，ctr 是无符号类型
// 正确：find_char 的第一个形参是对常量的引用
find_char("Hello World!", 'o', ctr);
```

要想调用引用版本的 reset（参见 6.2.2 节，第 189 页），只能使用 int 类型的对象，而不能使用字面值、求值结果为 int 的表达式、需要转换的对象或者 const int 类型的对象。类似的，要想调用指针版本的 reset（参见 6.2.1 节，第 188 页）只能使用 int*。

另一方面，我们能传递一个字符串字面值作为 `find_char`（参见 6.2.2 节，第 189 页）的第一个实参，这是因为该函数的引用形参是常量引用，而 C++ 允许我们用字面值初始化常量引用。

尽量使用常量引用

214

把函数不会改变的形参定义成（普通的）引用是一种比较常见的错误，这么做带给函数的调用者一种误导，即函数可以修改它的实参的值。此外，使用引用而非常量引用也会极大地限制函数所能接受的实参类型。就像刚刚看到的，我们不能把 `const` 对象、字面值或者需要类型转换的对象传递给普通的引用形参。

这种错误绝不像看起来那么简单，它可能造成出人意料的后果。以 6.2.2 节（第 189 页）的 `find_char` 函数为例，那个函数（正确地）将它的 `string` 类型的形参定义成常量引用。假如我们把它定义成普通的 `string&`：

```
//不良设计：第一个形参的类型应该是 const string&
string::size_type find_char(string &s, char c,
                             string::size_type &occurs);
```

则只能将 `find_char` 函数作用于 `string` 对象。类似下面这样的调用

```
find_char("Hello World", 'o', ctr);
```

将在编译时发生错误。

还有一个更难察觉的问题，假如其他函数（正确地）将它们的形参定义成常量引用，那么第二个版本的 `find_char` 无法在此类函数中正常使用。举个例子，我们希望在判断 `string` 对象是否是句子的函数中使用 `find_char`：

```
bool is_sentence(const string &s)
{
    // 如果在 s 的末尾有且只有一个句号，则 s 是一个句子
    string::size_type ctr = 0;
    return find_char(s, '.', ctr) == s.size() - 1 && ctr == 1;
}
```

如果 `find_char` 的第一个形参类型是 `string&`，那么上面这条调用 `find_char` 的语句将在编译时发生错误。原因在于 `s` 是常量引用，但 `find_char` 被（不正确地）定义成只能接受普通引用。

解决该问题的一种思路是修改 `is_sentence` 的形参类型，但是这么做只不过转移了错误而已，结果是 `is_sentence` 函数的调用者只能接受非常量 `string` 对象了。

正确的修改思路是改正 `find_char` 函数的形参。如果实在不能修改 `find_char`，就在 `is_sentence` 内部定义一个 `string` 类型的变量，令其为 `s` 的副本，然后把这个 `string` 对象传递给 `find_char`。

6.2.3 节练习

练习 6.16：下面的这个函数虽然合法，但是不算特别有用。指出它的局限性并设法改善。

```
bool is_empty(string& s) { return s.empty(); }
```

练习 6.17：编写一个函数，判断 `string` 对象中是否含有大写字母。编写另一个函数，把 `string` 对象全都改成小写形式。在这两个函数中你使用的形参类型相同吗？为什么？

练习 6.18: 为下面的函数编写函数声明，从给定的名字中推测函数具备的功能。

- (a) 名为 `compare` 的函数，返回布尔值，两个参数都是 `matrix` 类的引用。
- (b) 名为 `change_val` 的函数，返回 `vector<int>` 的迭代器，有两个参数：一个是 `int`，另一个是 `vector<int>` 的迭代器。

练习 6.19: 假定有如下声明，判断哪个调用合法、哪个调用不合法。对于不合法的函数调用，说明原因。

```
double calc(double);
int count(const string &, char);
int sum(vector<int>::iterator, vector<int>::iterator, int);
vector<int> vec(10);
(a) calc(23.4, 55.1);      (b) count("abcda", 'a');
(c) calc(66);              (d) sum(vec.begin(), vec.end(), 3.8);
```

练习 6.20: 引用形参什么时候应该是常量引用？如果形参应该是常量引用，而我们将其设为了普通引用，会发生什么情况？

6.2.4 数组形参

数组的两个特殊性质对我们定义和使用作用在数组上的函数有影响，这两个性质分别是：不允许拷贝数组（参见 3.5.1 节，第 102 页）以及使用数组时（通常）会将其转换成指针（参见 3.5.3 节，第 105 页）。因为不能拷贝数组，所以我们无法以值传递的方式使用数组参数。因为数组会被转换成指针，所以当我们为函数传递一个数组时，实际上传递的是指向数组首元素的指针。

尽管不能以值传递的方式传递数组，但是我们可以把形参写成类似数组的形式：

```
// 尽管形式不同，但这三个 print 函数是等价的
// 每个函数都有一个 const int* 类型的形参
void print(const int*);
void print(const int[]);      // 可以看出来，函数的意图是作用于一个数组
void print(const int[10]);    // 这里的维度表示我们期望数组含有多少元素，实际
                              // 不一定
```

215

尽管表现形式不同，但上面的三个函数是等价的：每个函数的唯一形参都是 `const int*` 类型的。当编译器处理对 `print` 函数的调用时，只检查传入的参数是否是 `const int*` 类型：

```
int i = 0, j[2] = {0, 1};
print(&i);          // 正确：&i 的类型是 int*
print(j);           // 正确：j 转换成 int* 并指向 j[0]
```

如果我们传给 `print` 函数的是一个数组，则实参自动地转换成指向数组首元素的指针，数组的大小对函数的调用没有影响。



WARNING

和其他使用数组的代码一样，以数组作为形参的函数也必须确保使用数组时不会越界。

因为数组是以指针的形式传递给函数的，所以一开始函数并不知道数组的确切尺寸，调用者应该为此提供一些额外的信息。管理指针形参有三种常用的技术。

216

使用标记指定数组长度

管理数组实参的第一种方法是要求数组本身包含一个结束标记,使用这种方法的典型示例是C风格字符串(参见3.5.4节,第109页)。C风格字符串存储在字符数组中,并且在最后一个字符后面跟着一个空字符。函数在处理C风格字符串时遇到空字符停止:

```
void print(const char *cp)
{
    if (cp)                // 若 cp 不是一个空指针
        while (*cp)        // 只要指针所指的字符不是空字符
            cout << *cp++; // 输出当前字符并将指针向前移动一个位置
}
```

这种方法适用于那些有明显结束标记且该标记不会与普通数据混淆的情况,但是对于像int这样所有取值都是合法值的数据就不太有效了。

使用标准库规范

管理数组实参的第二种技术是传递指向数组首元素和尾后元素的指针,这种方法受到了标准库技术的启发,关于其细节将在第II部分详细介绍。使用该方法,我们可以按照如下形式输出元素内容:

```
void print(const int *beg, const int *end)
{
    // 输出 beg 到 end 之间 (不含 end) 的所有元素
    while (beg != end)
        cout << *beg++ << endl; // 输出当前元素并将指针向前移动一个位置
}
```

while循环使用解引用运算符和后置递减运算符(参见4.5节,第131页)输出当前元素并在数组内将beg向前移动一个元素,当beg和end相等时结束循环。

为了调用这个函数,我们需要传入两个指针:一个指向要输出的首元素,另一个指向尾元素的下一位置:

```
int j[2] = {0, 1};
// j 转换成指向它首元素的指针
// 第二个实参是指向 j 的尾后元素的指针
print(begin(j), end(j)); // begin 和 end 函数, 参见第 3.5.3 节 (106 页)
```

只要调用者能正确地计算指针所指的位置,那么上述代码就是安全的。在这里,我们使用标准库begin和end函数(参见3.5.3节,第106页)提供所需的指针。

217 显式传递一个表示数组大小的形参

第三种管理数组实参的方法是专门定义一个表示数组大小的形参,在C程序和过去的C++程序中常常使用这种方法。使用该方法,可以将print函数重写成如下形式:

```
// const int ia[] 等价于 const int* ia
// size 表示数组的大小, 将它显式地传给函数用于控制对 ia 元素的访问
void print(const int ia[], size_t size)
{
    for (size_t i = 0; i != size; ++i) {
        cout << ia[i] << endl;
    }
}
```


这个版本的程序通过形参 `size` 的值确定要输出多少个元素，调用 `print` 函数时必须传入这个表示数组大小的值：

```
int j[] = { 0, 1 }; // 大小为 2 的整型数组
print(j, end(j) - begin(j));
```

只要传递给函数的 `size` 值不超过数组实际的大小，函数就是安全的。

数组形参和 `const`

我们的三个 `print` 函数都把数组形参定义成了指向 `const` 的指针，6.2.3 节（第 191 页）关于引用的讨论同样适用于指针。当函数不需要对数组元素执行写操作的时候，数组形参应该是指向 `const` 的指针（参见 2.4.2 节，第 56 页）。只有当函数确实要改变元素值的时候，才把形参定义成指向非常量的指针。

数组引用形参

C++ 语言允许将变量定义成数组的引用（参见 3.5.1 节，第 101 页），基于同样的道理，形参也可以是数组的引用。此时，引用形参绑定到对应的实参上，也就是绑定到数组上：

```
// 正确：形参是数组的引用，维度是类型的一部分
void print(int (&arr)[10])
{
    for (auto elem : arr)
        cout << elem << endl;
}
```

`&arr` 两端的括号必不可少（参见 3.5.1 节，第 101 页）：



```
f(int &arr[10])    // 错误：将 arr 声明成了引用的数组
f(int (&arr)[10]) // 正确：arr 是具有 10 个整数的整型数组的引用
```

因为数组的大小是构成数组类型的一部分，所以只要不超过维度，在函数体内就可以放心地使用数组。但是，这一用法也无形中限制了 `print` 函数的可用性，我们只能将函数作用于大小为 10 的数组：

218

```
int i = 0, j[2] = {0, 1};
int k[10] = {0,1,2,3,4,5,6,7,8,9};
print(&i);           // 错误：实参不是含有 10 个整数的数组
print(j);            // 错误：实参不是含有 10 个整数的数组
print(k);            // 正确：实参是含有 10 个整数的数组
```

16.1.1 节（第 578 页）将要介绍我们应该如何编写这个函数，使其可以给引用类型的形参传递任意大小的数组。

传递多维数组

我们曾经介绍过，在 C++ 语言中实际上没有真正的多维数组（参见 3.6 节，第 112 页），所谓多维数组其实是数组的数组。

和所有数组一样，当将多维数组传递给函数时，真正传递的是指向数组首元素的指针（参见 3.6 节，第 115 页）。因为我们处理的是数组的数组，所以首元素本身就是一个数组，指针就是一个指向数组的指针。数组第二维（以及后面所有维度）的大小都是数组类型的一部分，不能省略：

```
// matrix 指向数组的首元素，该数组的元素是由 10 个整数构成的数组
void print(int (*matrix)[10], int rowSize) { /* ... */ }
```

上述语句将 matrix 声明成指向含有 10 个整数的数组的指针。



再一次强调，*matrix 两端的括号必不可少：

```
int *matrix[10];           // 10 个指针构成的数组
int (*matrix)[10];         // 指向含有 10 个整数的数组的指针
```

我们也可以使用数组的语法定义函数，此时编译器会一如既往地忽略掉第一个维度，所以最好不要把它包括在形参列表内：

```
// 等价定义
void print(int matrix[][10], int rowSize) { /* ... */ }
```

matrix 的声明看起来是一个二维数组，实际上形参是指向含有 10 个整数的数组的指针。

6.2.4 节练习

练习 6.21: 编写一个函数，令其接受两个参数：一个是 int 型的数，另一个是 int 指针。函数比较 int 的值和指针所指的数，返回较大的那个。在该函数中指针的类型应该是什么？

练习 6.22: 编写一个函数，令其交换两个 int 指针。

练习 6.23: 参考本节介绍的几个 print 函数，根据理解编写你自己的版本。依次调用每个函数使其输入下面定义的 i 和 j：

```
int i = 0, j[2] = {0, 1};
```

练习 6.24: 描述下面这个函数的行为。如果代码中存在问题，请指出并改正。

```
void print(const int ia[10])
{
    for (size_t i = 0; i != 10; ++i)
        cout << ia[i] << endl;
}
```

6.2.5 main: 处理命令行选项

main 函数是演示 C++ 程序如何向函数传递数组的好例子。到目前为止，我们定义的 main 函数都只有空形参列表：

```
int main() { ... }
```

然而，有时我们确实需要给 main 传递实参，一种常见的情况是用户通过设置一组选项来确定函数所要执行的操作。例如，假定 main 函数位于可执行文件 prog 之内，我们可以向程序传递下面的选项：

219 `prog -d -o ofile data0`

这些命令行选项通过两个（可选的）形参传递给 main 函数：

```
int main(int argc, char *argv[]) { ... }
```

第二个形参 `argv` 是一个数组，它的元素是指向 C 风格字符串的指针；第一个形参 `argc` 表示数组中字符串的数量。因为第二个形参是数组，所以 `main` 函数也可以定义成：

```
int main(int argc, char **argv) { ... }
```

其中 `argv` 指向 `char*`。

当实参传给 `main` 函数之后，`argv` 的第一个元素指向程序的名字或者一个空字符串，接下来的元素依次传递命令行提供的实参。最后一个指针之后的元素值保证为 0。

以上面提供的命令行为例，`argc` 应该等于 5，`argv` 应该包含如下的 C 风格字符串：

```
argv[0] = "prog";    // 或者 argv[0] 也可以指向一个空字符串
argv[1] = "-d";
argv[2] = "-o";
argv[3] = "ofile";
argv[4] = "data0";
argv[5] = 0;
```



当使用 `argv` 中的实参时，一定要记得可选的实参从 `argv[1]` 开始；`argv[0]` 保存程序的名字，而非用户输入。

6.2.5 节练习

220

练习 6.25: 编写一个 `main` 函数，令其接受两个实参。把实参的内容连接成一个 `string` 对象并输出出来。

练习 6.26: 编写一个程序，使其接受本节所示的选项；输出传递给 `main` 函数的实参的内容。

6.2.6 含有可变形参的函数

有时我们无法提前预知应该向函数传递几个实参。例如，我们想要编写代码输出程序产生的错误信息，此时最好用同一个函数实现该项功能，以便对所有错误的处理能够整齐划一。然而，错误信息的种类不同，所以调用错误输出函数时传递的实参也各不相同。

为了编写能处理不同数量实参的函数，C++11 新标准提供了两种主要的方法：如果所有的实参类型相同，可以传递一个名为 `initializer_list` 的标准库类型；如果实参的类型不同，我们可以编写一种特殊的函数，也就是所谓的可变参数模板，关于它的细节将在 16.4 节（第 618 页）介绍。

C++ 还有一种特殊的形参类型（即省略符），可以用它传递可变数量的实参。本节将简要介绍省略符形参，不过需要注意的是，这种功能一般只用于与 C 函数交互的接口程序。

`initializer_list` 形参

如果函数的实参数量未知但是全部实参的类型都相同，我们可以使用 `initializer_list` 类型的形参。`initializer_list` 是一种标准库类型，用于表示某种特定类型的值的数组（参见 3.5 节，第 101 页）。`initializer_list` 类型定义在同名的头文件中，它提供的操作如表 6.1 所示。

**C++
11**

表 6.1: initializer_list 提供的操作

<code>initializer_list<T> lst;</code>	默认初始化: T 类型元素的空列表
<code>initializer_list<T> lst{a,b,c...};</code>	lst 的元素数量和初始值一样多; lst 的元素是对应初始值的副本; 列表中的元素是 const
<code>lst2(lst)</code> <code>lst2 = lst</code>	拷贝或赋值一个 <code>initializer_list</code> 对象不会拷贝列表中的元素; 拷贝后, 原始列表和副本共享元素
<code>lst.size()</code>	列表中的元素数量
<code>lst.begin()</code>	返回指向 lst 中首元素的指针
<code>lst.end()</code>	返回指向 lst 中尾元素下一位置的指针

221

和 `vector` 一样, `initializer_list` 也是一种模板类型 (参见 3.3 节, 第 86 页)。定义 `initializer_list` 对象时, 必须说明列表中所含元素的类型:

```
initializer_list<string> ls; // initializer_list 的元素类型是 string
initializer_list<int> li;   // initializer_list 的元素类型是 int
```

和 `vector` 不一样的是, `initializer_list` 对象中的元素永远是常量值, 我们无法改变 `initializer_list` 对象中元素的值。

我们使用如下的形式编写输出错误信息的函数, 使其可以作用于可变数量的实参:

```
void error_msg(initializer_list<string> il)
{
    for (auto beg = il.begin(); beg != il.end(); ++beg)
        cout << *beg << " ";
    cout << endl;
}
```

作用于 `initializer_list` 对象的 `begin` 和 `end` 操作类似于 `vector` 对应的成员 (参见 3.4.1 节, 第 195 页)。`begin()` 成员提供一个指向列表首元素的指针, `end()` 成员提供一个指向列表尾后元素的指针。我们的函数首先初始化 `beg` 令其表示首元素, 然后依次遍历列表中的每个元素。在循环体中, 解引用 `beg` 以访问当前元素并输出它的值。

如果想向 `initializer_list` 形参中传递一个值的序列, 则必须把序列放在一对花括号内:

```
// expected 和 actual 是 string 对象
if (expected != actual)
    error_msg({"functionX", expected, actual});
else
    error_msg({"functionX", "okay"});
```

在上面的代码中我们调用了同一个函数 `error_msg`, 但是两次调用传递的参数数量不同: 第一次调用传入了三个值, 第二次调用只传入了两个。

含有 `initializer_list` 形参的函数也可以同时拥有其他形参。例如, 调试系统可能有个名为 `ErrCode` 的类用来表示不同类型的错误, 因此我们可以改写之前的程序, 使其包含一个 `initializer_list` 形参和一个 `ErrCode` 形参:

```
void error_msg(ErrCode e, initializer_list<string> il)
{
```

```
cout << e.msg() << ": ";  
for (const auto &elem : il)  
    cout << elem << " ";  
cout << endl;  
}
```

因为 `initializer_list` 包含 `begin` 和 `end` 成员,所以我们可以使用范围 `for` 循环(参见 5.4.3 节,第 167 页)处理其中的元素。和之前的版本类似,这段程序遍历传给 `il` 形参的列表值,每次迭代时访问一个元素。

为了调用这个版本的 `error_msg` 函数,需要额外传递一个 `ErrCode` 实参:

222

```
if (expected != actual)  
    error_msg(ErrCode(42), {"functionX", expected, actual});  
else  
    error_msg(ErrCode(0), {"functionX", "okay"});
```

省略符形参



省略符形参是为了便于 C++ 程序访问某些特殊的 C 代码而设置的,这些代码使用了名为 `varargs` 的 C 标准库功能。通常,省略符形参不应用于其他目的。你的 C 编译器文档会描述如何使用 `varargs`。



省略符形参应该仅仅用于 C 和 C++ 通用的类型。特别应该注意的是,大多数类类型的对象在传递给省略符形参时都无法正确拷贝。

省略符形参只能出现在形参列表的最后一个位置,它的形式无外乎以下两种:

```
void foo(parm_list, ...);  
void foo(...);
```

第一种形式指定了 `foo` 函数的部分形参的类型,对应于这些形参的实参将会执行正常的类型检查。省略符形参所对应的实参无须类型检查。在第一种形式中,形参声明后面的逗号是可选的。

6.2.6 节练习

练习 6.27: 编写一个函数,它的参数是 `initializer_list<int>` 类型的对象,函数的功能是计算列表中所有元素的和。

练习 6.28: 在 `error_msg` 函数的第二个版本中包含 `ErrCode` 类型的参数,其中循环内的 `elem` 是什么类型?

练习 6.29: 在范围 `for` 循环中使用 `initializer_list` 对象时,应该将循环控制变量声明成引用类型吗?为什么?

6.3 返回类型和 return 语句

`return` 语句终止当前正在执行的函数并将控制权返回到调用该函数的地方。`return` 语句有两种形式:

```
return;  
return expression;
```